

Let's start by discussing **Loops**. A loop in assembly is a set of instructions that repeat for **rcx** times. Let's take the following example:

```

exampleLoop:
    instruction 1
    instruction 2
    instruction 3
    instruction 4
    instruction 5
    loop exampleLoop

```

Once the assembly code reaches `exampleLoop`, it will start executing the instructions under it. We should set the number of iterations we want the loop to go through in the `rcx` register. Every time the loop reaches the `loop` instruction, it will decrease `rcx` by `1` (i.e., `dec rcx`) and jump back to the specified label, `exampleLoop` in this case. So, before we enter any loop, we should `mov` the number of loop iterations we want to the `rcx` register.

Instruction	Description	Example
<code>mov rcx, x</code>	Sets loop (<code>rcx</code>) counter to <code>x</code>	<code>mov rcx, 3</code>
<code>loop</code>	Jumps back to the start of <code>loop</code> until counter reaches <code>0</code>	<code>loop exampleLoop</code>

loopFib

To demonstrate this, let's go back to our `fib.s` code:

```

global _start

```

```
section .text
_start:
    xor rax, rax
    xor rbx, rbx
    inc rbx
    add rax, rbx
```

Since any current Fibonacci number is the sum of the two numbers preceding it, we can automate this with a loop. Let's assume that the current number is stored in `rax`, so it is F_n , and the next number is stored in `rbx`, so it is F_{n+1} .

Starting with the last number as `0` and the current number as `1`, we can have our loop as follows:

1. Get next number with $0 + 1 = 1$
2. Move the current number to the last number (`1` in place of `0`)
3. Move the next number to the current number (`1` in place of `1`)
4. Loop

If we do this, we'll end up with `1` as the last number and `1` as the current number. If we loop again, we'll get `1` as the last number and `2` as the current number. So, let's implement this as assembly instructions. Since we can discard the last number `0` after we use it in the add, let's store the result in its place:

- `add rax, rbx`

We need to move the current number to the last number's place and move the following number to the current number. However, we have the following number in `rax` and the now old number

in `rbx`, so they are swapped. Can you think of any instruction to swap them?

Let's use the `xchg` instruction to swap both numbers:

- `xchg rax, rbx`

Now we can simply `loop`. However, before we enter a loop, we should set `rcx` to the count of iterations we want. Let's start with `10` iterations and add it after initializing the `rax` and `rbx` to `0` and `1`:

```
nasml
_start:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
    mov rcx, 10
```

Now we can define our loop as discussed above:

```
nasml
loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    loop loopFib
```

So, our final code is:

```

global _start

section .text
_start:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
    mov rcx, 10
loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    loop loopFib

```

Loop loopFib

Let's assemble our code, and run it with `gdb`. We'll break at `b loopFib`, so that we can examine the code at each iteration of the loop. We see the following register values before the first iteration:

```

$ ./assembler.sh fib.s -g
gef➤ b loopFib
Breakpoint 1 at 0x40100e
gef➤ r

```

```

registers ——— $rax : 0x0
$rbx : 0x1
$rcx : 0xa

```

We see that we start with `rax = 0` and `rbx = 1`. Let's press `c` to continue to the next iteration:

```
registers — $rax : 0x1
$rbx      : 0x1
$rcx      : 0x9
```

Now we have `1` and `1`, as expected, with `9` iterations left. Let's `c` ontinue again:

```
registers — $rax : 0x1
$rbx      : 0x2
$rcx      : 0x8
```

Now we are at `1` and `2`. Let's check the next three iterations:

```
registers — $rax : 0x2
$rbx      : 0x3
$rcx      : 0x7
```

```
registers — $rax : 0x3
$rbx      : 0x5
$rcx      : 0x6
```

```
registers — $rax : 0x5
$rbx      : 0x8
$rcx      : 0x5
```

As we can see, the script is successfully calculating the Fibonacci Sequence as 0, 1, 1, 2, 3, 5, 8. Let's continue to the last iteration, where `rbx` should be 55:

```
registers ——— $rax : 0x22
$rbx      : 0x37
$rcx      : 0x1
```

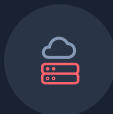
We see that `rbx` is 0x37, equal to 55 in decimal. We can confirm that with the `p/d $rbx` command:

```
gef> p/d $rbx
```

```
$3 = 55
```

As we can see, we have successfully used loops to automate the calculation of the Fibonacci Sequence. Try increasing `rcx` to see what are the next numbers in the Fibonacci Sequence.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.

 Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions 

PRO

 Answer questions in English to ensure an accurate response.

Question 1

 +1

 +40



Edit the attached assembly code to loop the "loop" label 5 times. What is the hex value of "rax" by the end?

Write your answer

 Submit



Hint



Download File

← Previous

Section 13 / 24

 +20

Next

Mark Complete & Next